

Dokumentacja z projektu

Programowanie Obiektowe

System zarządzania zawodami podnoszenia ciężarów

Backend: Spring Boot 3 / Java 21 | Frontend: Angular 21 / TypeScript

Imię i nazwisko: Wojciech Górka
Wydział: WIEiK
Kierunek: IwIK
Grupa: P1

1. Wstęp

Projekt jest aplikacją webową wspomagającą organizację i przebieg zawodów podnoszenia ciężarów. Umożliwia zarządzanie zdarzeniami (event), zawodnikami, sędziami i trenerami oraz rejestrowanie podejść (attempt) wraz z decyzjami sędziowskimi wyznaczającymi wynik (valid / no-lift / pending).

Stos technologiczny obejmuje Spring Boot 3 z Java 21 po stronie serwera oraz Angular 21 z TypeScript po stronie klienta. Komunikacja odbywa się przez REST API z serializacją JSON.

Architektura aplikacji jest trójwarstwowa: warstwa prezentacji (Angular), warstwa logiki biznesowej (Spring Services) oraz warstwa dostępu do danych (Spring Data JPA / PostgreSQL). Podział ten wymusza enkapsulację każdej warstwy i komunikację wyłącznie przez zdefiniowane interfejsy (DTO, Observables).

2. Model dziedziny - Backend

2.1 Hierarchia Person

Klasy Coach, Competitor i Judge dziedziczą po abstrakcyjnej encji Person. Zastosowano strategię JOINED TABLE. każda podklasa ma własną tabelę połączoną kluczem obcym z tabelą person. Pozwala to na normalizację danych i polimorficzne zapytania JPA.

Fragment nr 1 - klasa Person i podklasa Coach (dziedziczenie + @SuperBuilder)

```
// Person.java - klasa bazowa z @Inheritance(JOINED)
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
@SuperBuilder @Getter @Setter @NoArgsConstructor @AllArgsConstructor
@Table(name = "person")
```

```

public abstract class Person {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Column(nullable = false) private String firstName;
    @Column(nullable = false) private String lastName;
    private LocalDate dateOfBirth;
}

// Coach.java - podklasa z własnym polem i relacją
@Entity @SuperBuilder @Table(name = "coach")
public class Coach extends Person {
    private String team;
    @OneToMany(mappedBy = "coach", fetch = FetchType.LAZY)
    private List<Competitor> competitors;
}

```

Adnotacja `@SuperBuilder` (Lombok) umożliwia wywołanie buildera przez całą hierarchię klas. Strategia `InheritanceType.JOINED` pozwala przechowywać wspólne pola raz w tabeli `person`, a pola specyficzne w tabelach `coach`, `judge` i `competitor`.

2.2 Encje Event i Attempt - relacje JPA

Event łączy się z listą `Competitor` i `Judge` przez relacje `@ManyToMany` (tabele pośrednie `event_competitors`, `event_judges`). `Attempt` agreguje decyzje sędziowskie (`JudgeDecision`) w relacji `@OneToMany` z `CascadeType.ALL` i `orphanRemoval = true`, co zapewnia automatyczne usuwanie osieroconych decyzji.

Fragment nr 2 - relacje ManyToMany w Event i OneToMany w Attempt

```

// Event.java
@ManyToMany
@JoinTable(name = "event_competitors")
private List<Competitor> competitors;

@ManyToMany
@JoinTable(name = "event_judges")
private List<Judge> judges;

// Attempt.java - kaskadowe zarządzanie decyzjami
@OneToMany(mappedBy = "attempt",
            cascade = CascadeType.ALL,
            orphanRemoval = true)
private List<JudgeDecision> judgeDecisions = new ArrayList<>();

```

3. Wzorce obiektowe - Backend

3.1 Repository Pattern (DAO)

Repozytoria rozszerzają interfejs `JpaRepository<T, UUID>`, uzyskując automatyczne implementacje operacji CRUD. Dla niestandardowych zapytań stosuje się adnotację `@Query` z JPQL i JOIN FETCH eliminującym problem N+1.

Fragment nr 3 - JPQL z JOIN FETCH w CompetitorRepo

```
// CompetitorRepo.java
public interface CompetitorRepo extends JpaRepository<Competitor, UUID> {

    @Query("SELECT c FROM Competitor c JOIN FETCH c.coach WHERE c.id = :id")
    Optional<Competitor> getCompetitorWithCoach(@Param("id") UUID id);

    @Query("SELECT c FROM Competitor c JOIN FETCH c.coach")
    List<Competitor> getAllCompetitorsWithCoaches();
}
```

3.2 Builder Pattern

Klasy encji korzystają z `@Builder` (lub `@SuperBuilder` w hierarchii), co pozwala tworzyć obiekty deklaratywnie zamiast wieloparametrowych konstruktorów. Szczególnie widoczne w `AttemptService` przy tworzeniu nowego `Attempt` i `JudgeDecision`.

Fragment nr 4 - Builder w AttemptService

```
// AttemptService.java - warunkowe tworzenie obiektu builderem
Attempt attempt = attemptRepo
    .findByEvent_IdAndCompetitor_IdAndDisciplineAndAttemptNumber(...)
    .orElseGet(() -> Attempt.builder()
        .event(event)
        .competitor(competitor)
        .discipline(body.getDiscipline())
        .attemptNumber(body.getAttemptNumber())
        .build());

// JudgeDecision tworzony builderem w pętli
attempt.getJudgeDecisions().add(JudgeDecision.builder()
    .attempt(attempt).judge(judge).valid(d.isValid()).build());
```

3.3 Enkapsulacja warstw - DTO + Mapper

Encje nigdy nie są eksponowane bezpośrednio przez API. Serwisy przyjmują obiekty `Request` i zwracają obiekty `Response` (DTO). Konwersja odbywa się w dedykowanych klasach `Mapper`, co izoluje model wewnętrzny od kontraktu HTTP.

Fragment nr 5 - EventMapper: oddzielenie encji od DTO

```
// EventMapper.java
public EventResponse toResponse(Event event) {
    return EventResponse.builder()
        .id(event.getId())
        .name(event.getName())
        .rank(event.getRank())
        .location(event.getLocation())
        .startDate(event.getStartDate())
        .build();
}
```

3.4 Dependency Injection (@RequiredArgsConstructor)

Spring wstrzykuje zależności przez konstruktor - `@RequiredArgsConstructor` (Lombok) generuje konstruktor dla wszystkich pól final. Eliminuje boilerplate i ułatwia testowanie jednostkowe przez podmianę mocków.

Fragment nr 6 - DI w AttemptService

```
// AttemptService.java
@Service
@RequiredArgsConstructor
public class AttemptService {
    private final AttemptRepo attemptRepo;
    private final EventRepo eventRepo;
    private final CompetitorRepo competitorRepo;
    private final JudgeRepo judgeRepo;
    // Spring wstrzykuje wszystkie zależności przez konstruktor
}
```

3.5 Logika domenowa - computeResult

Metoda `computeResult` w `AttemptService` hermetyzuje regułę biznesową: podejście jest uznane jeśli co najmniej 2 z 3 sędziów zatwierdzi lift. Wynik jest wyliczany na żądanie, bez zapisu w bazie danych.

Fragment nr 7 - logika biznesowa wyznaczania wyniku

```
// AttemptService.java
private AttemptResult computeResult(List<JudgeDecision> decisions) {
    if (decisions.size() < 3) return AttemptResult.PENDING;
    long valid = decisions.stream()
        .filter(JudgeDecision::isValid)
        .count();
    return valid >= 2 ? AttemptResult.VALID : AttemptResult.NO_LIFT;
}
```

4. Model dziedziczny - Frontend (TypeScript)

4.1 Hierarchia klas User

Frontend odzwierciedla hierarchię domenową backendu za pomocą klas TypeScript z enkapsulacją pól przez prywatne atrybuty i publiczne getter/setter. Klasa `User` jest klasą bazową; `Coach`, `Judge` i `Competitor` ją rozszerzają.

Fragment nr 8 - klasa User i rozszerzająca Coach (TS)

```
// User.ts - klasa bazowa z prywatnymi polami i getterami
export class User {
    private _id: string;
    private _firstName: string;
    constructor(id: string, firstName: string, ...) { ... }
    public get id() { return this._id; }
}
```

```

    public get firstName() { return this._firstName; }
    public set firstName(newName: string) { this._firstName = newName; }
}

// Coach.ts - dziedziczenie i wywołanie super()
export class Coach extends User {
    private _team: string;
    constructor(id, firstName, lastName, dob, team) {
        super(id, firstName, lastName, dob);
        this._team = team;
    }
    public get team() { return this._team; }
    public set team(t: string) { this._team = t; }
}

```

4.2 Interfejsy Request/Response

Każdy typ użytkownika posiada parę interfejsów: XxxRequest (dane wysyłane do API) i XxxResponse (dane odbierane z API). Pozwala to TypeScript sprawdzać kształt danych w czasie kompilacji bez konieczności mapowania do klas.

Fragment nr 9 - interfejsy CoachRequest / CoachResponse

```

// Coach.ts
export interface CoachRequest {
    firstName: string;
    lastName: string;
    dateOfBirth: Date;
    team: string;
}
export interface CoachResponse {
    id: string; // pole readonly - zwracane przez serwer
    firstName: string;
    lastName: string;
    dateOfBirth: Date;
    team: string;
}

```

5. Wzorce obiektowe - Frontend

5.1 Singleton Service (@Injectable root)

Wszystkie serwisy są dekorowane @Injectable({ providedIn: 'root' }), co tworzy jedną instancję na całą aplikację. Jest to implementacja wzorca Singleton w Angular Dependency Injection.

5.2 Observable / BehaviorSubject - enkapsulacja stanu

UserService i EventService przechowują stan (listy zasobów) w prywatnych BehaviorSubject. Komponentom udostępniane są wyłącznie publiczne Observable (\$ suffix), przez co stan może być modyfikowany tylko przez metody serwisu.

Fragment nr 10 - BehaviorSubject w UserService

```
// user.service.ts
@Injectable({ providedIn: 'root' })
export class UserService {
  private coaches = new BehaviorSubject<CoachResponse[]>([]);
  private judges = new BehaviorSubject<JudgeResponse[]>([]);
  private competitors = new BehaviorSubject<CompetitorResponse[]>([]);

  // Tylko readonly stream - komponenty nie mogą mutować stanu
  public coaches$ = this.coaches.asObservable();
  public judges$ = this.judges.asObservable();
  public competitors$ = this.competitors.asObservable();
}
```

5.3 Kompozycja - EventManagerComponent

EventManagerComponent nie implementuje logiki HTTP, deleguje ją do wstrzykniętych serwisów EventService i AttemptService. Jest to kompozycja: komponent zarządza stanem widoku (sygnały Angular), a serwisy zarządzają danymi.

Fragment nr 11 - kompozycja serwisów w komponencie

```
// event-manager.component.ts
export class EventManagerComponent implements OnInit {
  private eventService = inject(EventService);
  private attemptService = inject(AttemptService);
  private alertService = inject(AlertService);

  // Stan widoku jako sygnały Angular (reactive primitives)
  protected event = signal<EventWithFullData | null>(null);
  protected attempts = signal<AttemptResponse[]>([]);
  protected activeSlot = signal<ActiveSlot | null>(null);
}
```

5.4 Lokalny update stanu bez przeładowania

Po zapisaniu podejścia komponent aktualizuje lokalną listę attempts bez ponownego wywołania API. Metoda signal.update() zamienia lub dołącza element na podstawie porównania klucza kompozytowego.

Fragment nr 12 - optymistyczna aktualizacja listy

```
// event-manager.component.ts - saveSlot()
this.attempts.update((list) => {
  const idx = list.findIndex(a =>
    a.competitorId === saved.competitorId &&
    a.discipline === saved.discipline &&
    a.attemptNumber === saved.attemptNumber,
  );
  return idx >= 0
    ? list.map((a, i) => (i === idx ? saved : a))
    : [...list, saved];
});
```

6. Komunikacja Frontend ↔ Backend

6.1 REST API i HttpClient

Frontend komunikuje się z backendem przez HTTP przy użyciu Angular HttpClient. Każde żądanie zwraca `Observable<T>`, strumień jest subskrybowany w komponencie lub metodzie serwisu z obsługą błędów przez `callback error`:

Przepływ danych - tworzenie nowego eventu

```
Angular Component
--> eventService.addEvent(eventRequest)
--> POST /api/event { name, rank, location, startDate, judges[],
competitors[] }

EventController.addEvent(@RequestBody EventRequest)
--> eventService.upsertEvent(body, null)
--> judgeRepo.findById() x N // walidacja sędziów
--> competitorRepo.findById() x N // walidacja zawodników
--> eventRepo.save(event) // zapis w PostgreSQL
--> mapper.toResponse(saved) // konwersja do DTO
<-- 201 Created { id, name, rank, location, startDate }

EventService (Angular) tap()
--> getAll().subscribe() // odświeżenie BehaviorSubject
--> events$ emits new list // widok automatycznie zaktualizowany
```

6.2 Walidacja po stronie serwera

`EventService` sprawdza minimalną liczbę sędziów (co najmniej 3) przed jakimkolwiek zapisem. Błędy biznesowe są zgłaszane przez `ResponseStatusException`, co Spring automatycznie mapuje na odpowiedzi 4xx.

```
// EventService.java - reguła biznesowa jako guard clause
if (body.getJudges() == null || body.getJudges().size() < 3) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST,
        "Event requires at least 3 judges");
}
```

7. Podsumowanie

Poniższa tabela zestawia zastosowane wzorce programowania obiektowego z ich lokalizacją w projekcie.

Wzorzec OOP	Warstwa	Przykład klasy / metody
Dziedziczenie	Backend	Person → Coach / Competitor / Judge (JOINED)

Wzorzec OOP	Warstwa	Przykład klasy / metody
Dziedziczenie	Frontend	User → Coach / Competitor (TypeScript)
Enkapsulacja	Backend	DTO (XxxRequest / XxxResponse) + EventMapper
Enkapsulacja	Frontend	BehaviorSubject + asObservable() w UserService
Builder Pattern	Backend	@Builder / @SuperBuilder - Attempt, JudgeDecision
Repository (DAO)	Backend	CompetitorRepo extends JpaRepository<Competitor, UUID>
Singleton	Frontend	@Injectable({ providedIn: 'root' }) - UserService
Dependency Injection	Backend	@RequiredArgsConstructor - AttemptService, EventService
Dependency Injection	Frontend	inject() - EventManagerComponent
Kompozycja	Frontend	EventManagerComponent agreguje EventService + AttemptService
Observer / Reactive	Frontend	BehaviorSubject + Observable\$ - EventService
Logika domenowa	Backend	AttemptService.computeResult() - reguła 2 z 3 sędziów

Projekt demonstruje zastosowanie paradygmatu obiektowego po obu stronach aplikacji: dziedziczenie hierarchii encji z polimorfizmem JPA po stronie serwera, enkapsulację stanu z reaktywnym przepływem danych po stronie klienta oraz warstwy serwisów i mapperów jako wyraźną separację odpowiedzialności.